

Modelagem de Risco de Crédito

Pipeline completo: EDA → Features → Modelagem → Interpretabilidade → Monitoramento

Hamilton Nascimento

Projeto de portfólio — Ciência de Dados aplicada a Crédito

Sobre o Projeto

Este projeto desenvolve um pipeline completo de modelagem de risco de crédito, cobrindo desde a análise exploratória dos dados até o monitoramento do modelo em produção simulada. O objetivo é construir um sistema capaz de prever a probabilidade de inadimplência grave de um cliente — definida como atraso de 90 dias ou mais em qualquer pagamento nos próximos 2 anos — e traduzir essa previsão em uma política de concessão de crédito com trade-offs explícitos.

O projeto cobre o ciclo completo que uma fintech ou banco digital executaria internamente: entendimento do negócio, análise dos dados, construção de features com lógica de negócio, modelagem comparativa, interpretabilidade com SHAP e simulação de diferentes políticas de aprovação.

Dataset

Give Me Some Credit — Kaggle. 150.000 registros de clientes com 10 variáveis preditoras e 1 variável-alvo binária (SeriousDlqin2yrs). Base com desbalanceamento real: ~6,7% de inadimplentes.

Ferramentas

Categoria	Tecnologias
Linguagem	Python 3.12+
Análise e manipulação	Pandas, NumPy
Visualização	Matplotlib, Seaborn
Modelagem	Scikit-learn, LightGBM, Optuna
Interpretabilidade	SHAP
Versionamento	Git + GitHub
IDE	Visual Studio Code

Métricas de Avaliação

O mercado de crédito usa métricas específicas — não apenas acurácia:

Métrica	O que mede	Por que usar
AUC-ROC	Capacidade de separar bons e maus pagadores	Métrica padrão do mercado de crédito

KS (Kolmogorov-Smirnov)	Máxima separação entre distribuições de scores	Mede o poder discriminativo do modelo
Gini	Versão normalizada do AUC ($\text{Gini} = 2 \times \text{AUC} - 1$)	Amplamente usado em relatórios de risco
Precision / Recall	Qualidade das previsões positivas	Avalia custo de errar para cada lado

Estrutura do Projeto — Sprints

O projeto está organizado em 6 sprints sequenciais. A ordem foi adaptada para trabalho em IDE (VS Code), iniciando pela exploração dos dados antes de estruturar o repositório.

Sprint 1	Análise Exploratória (EDA)	Semana 1
Inspeção inicial	Shape, tipos, primeiras linhas, proporção da variável-alvo.	
Missing values	Proporção de nulos por coluna. Investigar se o padrão tem relação com inadimplência.	
Distribuições e outliers	Histogramas para todas as variáveis. Identificar e documentar valores absurdos.	
Análise bivariada	Taxa de inadimplência por faixa de cada variável. Cada gráfico vira uma frase de insight.	
Correlações	Heatmap de correlação de Spearman. Identificar pares problemáticos.	
Arquivo de conclusões	eda_insights.md com 5-8 insights numerados com implicação de negócio para cada um.	
Entregáveis	eda.py rodando, figuras salvas em figures/ e eda_insights.md preenchido com decisões documentadas.	

Sprint 2	Preparação de Dados e Feature Engineering	Semanas 1–2
Tratamento de missings	Imputação por mediana. Criar flag binária para MonthlyIncome_missing.	
Tratamento de outliers	Winsorização nos percentis 1% e 99% nas variáveis identificadas na EDA.	
Criação de features	Mínimo 3 variáveis derivadas com justificativa de negócio: razão dívida/renda, flag de atraso grave, total de linhas abertas.	
Divisão treino/teste	80/20 com estratificação. Verificar proporção de inadimplentes em cada split.	
Desbalanceamento	Decidir estratégia: SMOTE, undersampling ou class_weight. Documentar a escolha.	
Arquivo de decisões	feature_decisions.md com cada decisão tomada e sua justificativa de negócio.	
Entregáveis	preprocessing.py rodando, datasets de treino e teste salvos e feature_decisions.md preenchido.	

Sprint 3	Estrutura do Repositório e Setup	Semana 2
Estrutura de pastas	Organizar src/, data/, reports/figures/, models/. Distribuir código existente nos arquivos corretos.	
Pipeline scikit-learn	Encapsular pré-processamento em Pipeline com ColumnTransformer. Evita vazamento de dados.	
config.py	Centralizar paths, seed, proporção de split e constantes. Nenhum número solto no código.	
GitHub + .gitignore	Repositório público. Excluir data/raw/, models/ e __pycache__/. Fixar versões com pip freeze.	
README inicial	Problema de negócio, dataset, estrutura do repositório e como rodar o projeto.	
Teste de reprodutibilidade	Clonar em pasta nova, instalar dependências e rodar do zero sem ajustes manuais.	
Entregáveis	Repositório público no GitHub com pipeline reproduzível, README inicial e histórico de commits limpo.	

Sprint 4	Modelagem e Avaliação	Semanas 3–4
Baseline: regressão logística	Treinar sem tuning. Avaliar com AUC-ROC, KS e Gini. Referência mínima que o modelo principal deve superar.	
Modelo principal: LightGBM	Validação cruzada estratificada (5-fold). Tuning com Optuna. Salvar modelo final com joblib.	
Tabela comparativa	Comparar todos os modelos com todas as métricas. Justificar escolha do modelo final em texto.	
Análise de threshold	Plotar curva ROC. Simular 3 pontos de corte e calcular impacto em volume e inadimplência esperada.	
Entregáveis	train.py e evaluate.py funcionando, modelo salvo em models/ e model_comparison.md com tabela e justificativa.	

Sprint 5	Interpretabilidade e Política de Crédito	Semana 4
SHAP global	Summary plot (beeswarm). Para as 5 variáveis mais importantes, escrever frase com implicação de negócio.	
SHAP individual	Waterfall plot para 1 cliente aprovado e 1 recusado. Explicação em linguagem de negócio para cada caso.	
Simulação de política	3 cenários de threshold (conservador, moderado, agressivo). Calcular volume de aprovações, taxa de inadimplência e perda estimada. Apresentar como tabela de trade-offs executiva.	
Entregáveis	policy.py com simulação, gráficos SHAP em reports/figures/ e tabela de cenários em policy_scenarios.md.	

Sprint 6	Monitoramento e Documentação Final	Semana 5
----------	------------------------------------	----------

PSI — Population Stability Index	Dividir dataset de teste em 2 períodos simulados. Calcular PSI por variável. Classificar: <0.1 estável, 0.1-0.2 alerta, >0.2 instável.
Score drift	Calcular PSI sobre a distribuição do score final nos dois períodos. Documentar ação recomendada se drift for detectado.
README final	Adicionar resultados reais (AUC, KS, Gini), threshold escolhido, link para relatório e print do SHAP mais relevante.
Relatório executivo	Documento de 2-3 páginas sem código: problema, solução, resultados, política recomendada e limitações.
Entregáveis	monitor.py com PSI e drift, README final completo e executive_summary.md. Repositório fechado e apresentável.

Cronograma Estimado

Com suporte ativo e dedicação de 2-3 horas por dia:

Sprint	Conteúdo	Duração estimada
1	Análise Exploratória (EDA)	3-4 dias
2	Preparação de dados e features	2-3 dias
3	Estrutura do repositório e setup	1 dia
4	Modelagem e avaliação	3-4 dias
5	Interpretabilidade e política	2-3 dias
6	Monitoramento e documentação	2 dias
Total		~2,5 semanas

Estimativa com suporte ativo via Claude. Sem suporte, estimativa de 4-5 semanas no mesmo regime de dedicação.

Por que este projeto é competitivo

Ciclo completo	A maioria dos projetos de portfólio para no modelo. Este cobre EDA, features, modelagem, interpretabilidade e monitoramento — o ciclo real de uma fintech.
Métricas do mercado	AUC, KS e Gini são as métricas que analistas de crédito usam. Usar acurácia seria um sinal de inexperiência.
SHAP com narrativa	Não basta calcular SHAP — é preciso traduzir em linguagem de negócio. Poucos projetos de portfólio fazem isso.
Simulação de política	Mostrar o trade-off entre threshold conservador e agressivo demonstra pensamento de negócio, não só técnico.
PSI e monitoramento	Quase nenhum projeto júnior chega até monitoramento de drift. Quem chega se destaca imediatamente.

